LeafOS Optional Node System

Advanced Learning Guide

v0.5.0

Scope note. This guide covers the optional SSH node subsystem. The current project-agent path is `leafctl live PATH`, backed by the version-2 typed inlet, resident supervisor, CPU-authoritative validation, and Python/native TUI. Read `DOCUMENTATION_MAP.md`, `ARCHITECTURE.md`, and `RESIDENT_STACK_USAGE.md` first for local project automation.

Contents

Architecture Overview

One binary, two roles

`leaf` is a single executable that branches on the command group invoked. The receiver engine (`core/node/leaf_node.py`) runs locally on every node. The distributor layer (`core/remote/remote.sh`) drives remote nodes by SSH-ing into them and invoking the same binary.

```
MASTER / distributor
  bin/leafctl --source--> core/remote/remote.sh
                                                          |
                                                LAN / WAN SSH
                                                          |
WORKER / receiver
  ssh -> bin/leafctl --source--> core/node/node.sh
                                                                    |
                                                          python3 leaf_node.
                                                              py
                                                                    |
  ~/.leaf/  inbox/  queued/  running/  done/  failed/
                        logs/TASK_ID.jsonl
                        results/TASK_ID/result.json
```

The master never contacts an HTTP API. It opens one SSH connection per operation and reads files the receiver wrote. That is the entire protocol.

Source chain

```
bin/leafctl
  source core/brand/brand.sh
  source core/log/log.sh
  source core/remote/remote.sh      # distributor layer
        source core/node/node.sh        # receiver wrapper
          resolves python3
          exports LEAF_NODE_PY path
```

`leaf_node.py` is loaded fresh per invocation. A node doing no work is a directory tree and an SSH daemon. No persistent process.

Workspace resolution

```
leaf_home() resolution order:
  1. --home FLAG
  2. $LEAF_HOME environment variable
  3. ~/.leaf  (default)
```

State Machine

Task lifecycle

```
                                                    +---- cancel flag ---+
                                                    |                    v
inbox/ -> queued/ -> running/ -> done/      failed/
                                    |
                                    +-> failed/   (non-zero exit)
```

State transitions are atomic file moves (`shutil.move`). A task occupies exactly one directory at all times. Crashes abandoning `running/` are detectable by `leaf task list -state running`.

Cooperative cancellation

The distributor creates `running/TASK_ID.cancel`. The runner polls for it. A runner that ignores the flag finishes normally — there is no `SIGKILL` without warning.

```
leaf task cancel gpu1 task_20260621_142203
# internally (over SSH):
# touch ~/.leaf/running/task_20260621_142203.cancel
```

In Python, check with:

```
if os.path.exists(_cancel_flag(home, task["task_id"])):
        return 130, "cancelled", {}
```

Writing a Custom Runner

A runner is a Python function:

```
def run_myrunner(home: str, task: dict, log) -> tuple:
        """
        home  -- absolute path to $LEAF_HOME
        task  -- parsed task JSON descriptor
        log   -- TaskLog; log.event("stdout", text="...")
        Returns (exit_code: int, status: str, output: dict)
        """
```

Minimal example: sleep runner

```
import time, os

def run_sleep(home, task, log):
        secs = float(task.get("payload", {}).get("seconds", 5))
        log.event("stdout", text="sleeping {}s".format(secs))
        time.sleep(secs)
        return 0, "ok", {"slept": secs}
```

Register in `RUNNERS` in `leaf_node.py`:

```
RUNNERS = {
        "echo": run_echo, "shell": run_shell,
        "hardware": run_hardware, "llama": run_llama,
        "sleep": run_sleep,   # add here
}
```

Streaming stdout with real-time cancellation

```
def run_myrunner(home, task, log):
        import subprocess
        proc = subprocess.Popen(
                ["long-running-cmd"],
                stdout=subprocess.PIPE, stderr=subprocess.STDOUT,
                universal_newlines=True,
        )
        for line in iter(proc.stdout.readline, ""):
                log.event("stdout", text=line.rstrip("\n"))
                if os.path.exists(_cancel_flag(home, task["task_id"])):
                        proc.terminate()
                        return 130, "cancelled", {}
        code = proc.wait()
        return code, "ok" if code == 0 else "error", {"exit_code": code}
```

The master sees each line in real time via `leaf task watch` (`tail -f` over SSH).

The Three Contracts

These schemas must stay stable. Internal rewrites and new runners are safe as long as these hold.

Task file schema

```
{
  "task_id":  "task_20260621_142203",  // stamped by distributor
  "kind":     "echo",                  // selects the runner
  "priority": 5,                       // 1 (low) - 10 (high)
  "payload":  { "message": "hello" }   // runner-defined
}
```

State directory names (frozen)

`inbox` `queued` `running` `done` `failed`

Do not rename. `_find_task()` iterates `STATE_DIRS` in order.

JSONL log event schema

field	type	always present
`time`	ISO-8601 string	yes
`event`	string	yes
`task_id`	string	yes
`runner`	string	`started` only
`text`	string	`stdout`/`stderr`
`exit_code`	int	terminal events

Standard sequence: `queued` → `started` → `stdout*` → `done` | `failed` | `cancelled`

Transport Layer Internals

Registry (nodes.json)

```
{
  "gpu1": {
        "target":    "user@192.168.1.44",
        "path":      "~/.leaf",
        "transport": "ssh"
  }
}
```

`_node_resolve NAME` in `remote.sh` reads this via `registry-get` (Python, no `jq` dependency), exporting `_RT`, `_RTGT`, `_RPATH`.

Transport dispatch in remote.sh

```
case "$_RT" in
        local) _local_engine task-start "$task_id" ;;
        ssh)   _ssh "$_RTGT" "leaf task start $task_id" ;;
esac
```

`_local_engine` is `LEAF_HOME="$_RPATH" python3 leaf_node.py`. This is why `tests/nodes.sh` needs no network.

SSH options

```
export LEAF_SSH_OPTS="-i ~/.ssh/leaf_node_key -p 2222"
```

Expanded by `_ssh()` and `_scp()` before all other arguments.

Multi-Node Patterns

Fan-out: submit to multiple nodes

```
for node in gpu1 gpu2 cpu1; do
        TASK_ID=$(leaf task submit "$node" examples/echo.task.json | tail -n1)
        echo "$node $TASK_ID"
done
```

Collect results from all workers

```
mkdir -p ./run_collection
for node in gpu1 gpu2; do
        leaf task pull "$node" "$TASK_ID" "./run_collection/$node"
done
```

Choose the idle node

```
pick_idle() {
        leaf nodes list --json | python3 -c "
import json, sys, subprocess
nodes = json.load(sys.stdin)
for name, cfg in nodes.items():
        r = subprocess.run(['leaf','nodes','ping',name],
                                        capture_output=True, text=True)
        d = json.loads(r.stdout) if r.returncode == 0 else {}
        if d.get('tasks_running', 1) == 0:
                print(name); break
"
}
TARGET=$(pick_idle)
leaf task submit "$TARGET" examples/shell.task.json
```

Security Hardening

Dedicated SSH key

```
ssh-keygen -t ed25519 -C "leaf-cluster" -f ~/.ssh/leaf_node_key
ssh-copy-id -i ~/.ssh/leaf_node_key.pub user@192.168.1.44
export LEAF_SSH_OPTS="-i ~/.ssh/leaf_node_key"
```

Restrict key to leaf commands only

In `~/.ssh/authorized_keys` on the worker:

```
command="leaf task start ${SSH_ORIGINAL_COMMAND#* }",
no-port-forwarding,no-X11-forwarding,no-agent-forwarding,no-pty
ssh-ed25519 AAAA... leaf-cluster
```

Firewall (ufw example)

```
ufw allow from 192.168.1.0/24 to any port 22
ufw deny 22
```

No other port needs to be open. No HTTP listener, no agent socket, no exposed API — only SSH.

Non-root dedicated user

```
useradd -m -s /bin/bash leafrunner
su - leafrunner -c "leaf node init --node-id worker1"
```

Debugging and Observability

Read JSONL logs directly

```
# all events
cat ~/.leaf/logs/TASK_ID.jsonl | python3 -m json.tool

# stdout lines only
grep '"event":"stdout"' ~/.leaf/logs/TASK_ID.jsonl \
  | python3 -c "
import sys, json
[print(json.loads(l)['text']) for l in sys.stdin]"
```

Orphaned running tasks

```
leaf task list --state running --json
leaf task cancel TASK_ID   # drops cancel flag; runner exits on next poll
```

LEAF_HOME isolation for debugging

```
LEAF_HOME=/tmp/debug_node leaf node init --node-id debug
LEAF_HOME=/tmp/debug_node leaf task start TASK_ID
```

SSH connectivity check

```
ssh $LEAF_SSH_OPTS user@192.168.1.44 "leaf node status --json"
ssh -v -o ConnectTimeout=5 user@192.168.1.44 "echo ok"
```

Extension Roadmap

Milestone	Planned change
0.6.0	`llama.cpp` runner; node selection by GPU presence
0.7.0	Priority queue; max-N concurrent tasks; queued drains by priority
0.8.0	`rsync` replaces `scp` for large/partial payloads
0.9.0	Restricted SSH command wrapper + `authorized_keys` templates
1.0.0	Optional read-only web viewer (`leaf node serve`), log streaming only

The three contracts in Section 4 are frozen from this milestone forward.

Full Command Reference

Receiver (runs on the worker)

```
leaf node init   [--node-id ID] [--role ROLE] [--json]
leaf node status [--json]
leaf node hardware [--json]
leaf task start   TASK_ID
leaf task list    [--state STATE] [--json]
leaf task logs    TASK_ID [--follow]
leaf task result  TASK_ID [--json]
leaf task cancel  TASK_ID
```

Distributor (runs on the master)

```
leaf nodes add   NAME USER@HOST [--transport ssh|local] [--path DIR]
leaf nodes list  [--json]
leaf nodes ping  NAME
leaf task submit NAME TASK_FILE
leaf task watch  NAME TASK_ID
leaf task pull   NAME TASK_ID [DEST]
leaf task cancel NAME TASK_ID
```

Key files

File	Purpose
`core/node/leaf_node.py`	Receiver engine (stdlib only)
`core/node/node.sh`	Brand-aware receiver wrapper
`core/remote/remote.sh`	Distributor + transport layer
`bin/leafctl`	CLI dispatch
`examples/*.task.json`	Example task descriptors
`tests/nodes.sh`	End-to-end local-transport test suite
`docs/NODES.md`	Architecture reference
`docs/QUICKSTART.md/.tex`	Quick start